Power BI

Interview Guide

DAX Functions, Key Facts, and Common Questions

Prepared for DTank54 Group
A1 English Level

Table of Contents

1. What is Power BI?

Power BI is a business intelligence tool made by Microsoft. It helps people see and understand their data. You can make reports, dashboards, and visual charts with it. Many companies use Power BI to make better decisions based on their data. Power BI has three main parts, and each part has a different job. Understanding these three parts is very important for your interview because interviewers almost always ask about them.

The Three Parts of Power BI

Power BI Desktop is a free application that you install on your computer. You use it to connect to data, clean the data, build data models, and create reports. Most of your work happens here. You can write DAX formulas, create relationships between tables, and design visual pages. When your report is ready, you publish it to the Power BI Service.

Power BI Service (also called Power BI Online) is a cloud-based service. You use a web browser to open it. Here you can share your reports with other people in your company. You can also set up automatic data refresh, create dashboards, and set permissions for who can see your reports. This is the place where teams work together on data.

Power BI Mobile is a mobile app for phones and tablets. You can view your dashboards and reports on your phone. It is useful when you are not at your computer but still need to check your data. The mobile app supports iOS and Android devices, and it keeps your reports interactive even on a small screen.

Part	Platform	Main Use	Cost
Power BI Desktop	Windows App	Build reports and models	Free
Power BI Service	Web Browser (Cloud)	Share and collaborate	Pro / Premium
Power BI Mobile	Phone / Tablet	View reports on the go	Free App

Table 1: Three Parts of Power BI

2. Power BI Architecture

Power BI follows a simple flow from data to visual report. This flow is important to understand because interviewers often ask you to explain how Power BI works from start to finish. The flow has four main steps, and each step is important. If you understand these steps well, you can answer many architecture questions in your interview.

The Four Steps of Power BI

- **Step 1 - Data Sources:** Power BI can connect to many data sources. Some examples are Excel files, SQL Server databases, SharePoint, web pages, and cloud services like Azure. You

can connect to one or many data sources at the same time. Power Query helps you connect and load data from these sources.

- **Step 2 - Data Modeling:** After you load data, you build a data model. This means you create relationships between tables, define calculated columns and measures, and organize your data so it is easy to use. A good data model is the foundation of a good report. Star schema is the most common pattern for data modeling in Power BI.
- **Step 3 - Data Visualization:** In this step, you create charts, tables, maps, and other visual elements on your report pages. Power BI has many visual types like bar charts, line charts, pie charts, maps, cards, and tables. You choose the best visual for your data story.
- **Step 4 - Share and Collaborate:** After you finish your report, you publish it to Power BI Service. Other people in your company can see it. You can also set up scheduled refresh so the data updates automatically. Dashboards and workspaces help you organize your reports.

Power Query vs DAX - When to Use What?

This is one of the most common interview questions. Power Query and DAX are two different tools inside Power BI, and they do different jobs. Power Query is for data transformation (cleaning and shaping data before it loads into the model). DAX is for calculations after data is loaded (creating measures and calculated columns for analysis). Power Query runs first when data is refreshed, and DAX runs later when users interact with the report. Understanding the difference between these two is very important.

Feature	Power Query (M)	DAX
Purpose	Data cleaning and shaping	Calculations and analysis
When it runs	At data refresh time	At query time (user interaction)
Language type	Functional language (M)	Expression language (DAX)
Main use	Filter rows, merge tables, change types	Measures, calculated columns
Where you write it	Power Query Editor	Modeling tab or formula bar
Row-level filtering	Yes (before load)	Yes (CALCULATE function)

Table 2: Power Query vs DAX Comparison

3. DAX Basics

DAX stands for Data Analysis Expressions. It is a formula language used in Power BI, Analysis Services, and Power Pivot in Excel. DAX helps you create custom calculations for your reports. You can create new columns (called calculated columns) and new values for visuals (called measures). DAX looks similar to Excel formulas, but it is much more powerful. It has over 250

functions and supports concepts like filter context and evaluation context that Excel does not have.

What Can DAX Do?

- â■¢ Create new columns in your tables based on formulas
- â■¢ Create measures that calculate values based on filter selections
- â■¢ Build complex calculations like year-to-date, running totals, and rankings
- â■¢ Filter and manipulate data in ways that visual filters cannot
- â■¢ Build time intelligence calculations (compare this month to last month)

DAX Syntax Basics

Every DAX formula starts with an equal sign (=), just like Excel. Then you write a function name, open parenthesis, put your arguments inside, and close parenthesis. Here is a simple example:

```
Total Sales = SUM(Sales[Amount])
```

In this example, "Total Sales" is the measure name. "SUM" is the DAX function. "Sales[Amount]" is the column we are adding up. The table name comes first, then the column name inside square brackets. This syntax pattern is the same for almost all DAX functions.

Calculated Columns vs Measures

This is one of the most important topics for Power BI interviews. Many interviewers ask you to explain the difference. A calculated column creates a new column in your table. It calculates a value for every row in that table. The result is stored in the table, and it uses memory. A measure does not create a column. Instead, it calculates a value when the user interacts with the report (like clicking a slicer or looking at a visual). Measures change based on the filter context. This is a key difference that you must understand.

Feature	Calculated Column	Measure
Where result lives	In the table (new column)	In memory (no column created)
When it calculates	At data refresh (once)	At query time (every interaction)
Uses memory?	Yes (stored in table)	No (calculated on the fly)
Filter context aware?	No (row-by-row only)	Yes (responds to filters)
Best for	Row-level categorization	Aggregations, ratios, KPIs
Can use in rows axis?	Yes	Yes
Performance impact	Increases model size	Faster (no storage needed)

Table 3: Calculated Column vs Measure

4. Important DAX Functions

In interviews, you need to know the most common DAX functions. You do not need to memorize all 250 functions, but you must know the important ones. Interviewers often ask you to write a DAX formula or explain what a function does. Below are the most important functions grouped by category. Study each one carefully and try to remember what it does and when to use it.

4.1 Aggregation Functions

Aggregation functions are the most basic and common DAX functions. They take a column of numbers and return a single value. SUM adds all numbers. AVERAGE calculates the mean. MIN returns the smallest value and MAX returns the largest. COUNT counts all rows with a value and COUNTROWS counts all rows in a table. These functions are the building blocks for more complex measures.

Function	Description	Example
SUM()	Adds all the numbers in a column	SUM(Sales[Amount])
AVERAGE()	Calculates the average of a column	AVERAGE(Sales[Price])
MIN()	Returns the smallest value in a column	MIN(Products[Price])
MAX()	Returns the largest value in a column	MAX(Products[Price])
COUNT()	Counts non-blank values in a column	COUNT(Sales[OrderID])
COUNTROWS()	Counts all rows in a table	COUNTROWS(Sales)
DISTINCTCOUNT()	Counts unique values in a column	DISTINCTCOUNT(Sales[CustomerID])

Table 4: Aggregation Functions

4.2 Filter Functions

Filter functions are very important in DAX. They let you control which rows are included in a calculation. CALCULATE is the most powerful and most asked-about DAX function. It changes the filter context for a calculation. FILTER returns a table of rows that meet a condition. ALL removes all filters from a column or table. These three functions are essential for interview preparation.

Function	Description	Example

CALCULATE()	Changes filter context for an expression. This is the most important DAX function. It can add, remove, or modify filters before calculating.	CALCULATE(SUM(Sales[Amount]), Sales[Year]=2024)
FILTER()	Returns a table with only the rows that meet a condition. Usually used inside other functions like CALCULATE.	FILTER(Sales, Sales[Amount] > 100)
ALL()	Removes all filters from a column or table. Very useful for percentage calculations and totals that ignore current filters.	CALCULATE(SUM(Sales[Amount]), ALL(Sales))
ALLEXCEPT()	Removes all filters except the ones you specify. Useful when you want to keep some filters and remove others.	CALCULATE(SUM(Sales[Amount]), ALLEXCEPT(Sales, Sales[Region]))
VALUES()	Returns a one-column table of unique values. Useful for checking what values are currently filtered in a column.	VALUES(Products[Category])
HASONEVALUE()	Returns TRUE if a column has exactly one value filtered. Often used in IF conditions for safety checks.	IF(HASONEVALUE(Products[Category]), ...)

Table 5: Filter Functions

4.3 Logical Functions

Logical functions help you make decisions in your DAX formulas. IF checks a condition and returns one value if true and another if false. SWITCH is like a multi-choice IF - it checks many conditions and returns the first match. These functions are very useful for categorizing data and building conditional calculations. Many interview questions require you to use IF or SWITCH in your answer.

Function	Description	Example
IF()	Checks a condition. Returns value A if TRUE, value B if FALSE.	IF(Sales[Amount] > 100, "High", "Low")
SWITCH()	Checks many conditions and returns the matching result. More readable than nested IFs.	SWITCH(Products[Category], "A", 1, "B", 2, 3)
AND()	Returns TRUE if ALL conditions are true.	IF(AND(Sales[Qty]>10, Sales[Price]>50), "Good", "Bad")
OR()	Returns TRUE if at least ONE condition is true.	IF(OR(Products[Color]="Red", Products[Color]="Blue"), "Popular", "Other")

COALESCE()	Returns the first non-blank value from a list of expressions.	COALESCE(Sales[Discount], 0)

Table 6: Logical Functions

4.4 Time Intelligence Functions

Time intelligence functions are a very common interview topic. These functions help you compare data across different time periods. For example, you can compare this month sales to last month sales, or calculate year-to-date totals. To use these functions, you need a date table in your model. A date table is a table with one row for each date and a continuous range of dates. Most interview questions about DAX will involve time intelligence.

Function	Description	Example
TOTALYTD()	Calculates year-to-date total from January 1 to the current filter date.	TOTALYTD(SUM(Sales[Amount]), Dates[Date])
TOTALQTD()	Calculates quarter-to-date total from the start of the quarter.	TOTALQTD(SUM(Sales[Amount]), Dates[Date])
TOTALMTD()	Calculates month-to-date total from the start of the month.	TOTALMTD(SUM(Sales[Amount]), Dates[Date])
SAMEPERIODLASTYEAR()	Returns dates from the same period last year for comparison.	CALCULATE(SUM(Sales), SAMEPERIODLASTYEAR(Dates[Date]))
DATEADD()	Shifts dates by a number of intervals (day, month, quarter, year).	CALCULATE(SUM(Sales), DATEADD(Dates[Date], -1, MONTH))
PREVIOUSDAY()	Returns the day before the current date in the filter context.	CALCULATE(SUM(Sales), PREVIOUSDAY(Dates[Date]))
PREVIOUSMONTH()	Returns all dates from the previous month.	CALCULATE(SUM(Sales), PREVIOUSMONTH(Dates[Date]))
DATESYTD()	Returns dates from the start of the year to the current date.	CALCULATE(SUM(Sales), DATESYTD(Dates[Date]))
DATESMTD()	Returns dates from the start of the month to the current date.	CALCULATE(SUM(Sales), DATESMTD(Dates[Date]))

Table 7: Time Intelligence Functions

4.5 Text and Information Functions

Text functions help you work with text data in your columns. CONCATENATE joins two text strings together. LEFT, RIGHT, and MID extract parts of a text string. UPPER and LOWER

change the case of text. SEARCH and FIND look for text inside other text. These functions are useful for cleaning data and creating meaningful labels in your reports.

Function	Description	Example
CONCATENATE()	Joins two text strings into one.	CONCATENATE(FirstName, " ", LastName)
LEFT() / RIGHT()	Returns the left/right N characters of a text.	LEFT(Product[Code], 3)
UPPER() / LOWER()	Converts text to uppercase or lowercase.	UPPER(Customer[City])
SEARCH() / FIND()	Finds the position of one text inside another.	SEARCH("-", Product[Code])
FORMAT()	Converts a value to text in a specified format.	FORMAT(Dates[Date], "MMM-YYYY")
REPLACE()	Replaces part of a text string with new text.	REPLACE(Phone, 1, 3, "+994")

Table 8: Text and Information Functions

4.6 Relationship and Table Functions

These functions help you work with relationships between tables and combine data from multiple tables. RELATED gets a value from a related table (many-to-one direction). RELATEDTABLE returns all rows from a related table. CROSSFILTER changes the filter direction of a relationship. USERELATIONSHIP lets you use an inactive relationship in a calculation. UNION and INTERSECT combine tables in different ways.

Function	Description	Example
RELATED()	Gets a value from a related table (follows many-to-one relationship).	RELATED(Products[CategoryName])
RELATEDTABLE()	Returns all rows from a related table (one-to-many direction).	COUNTROWS(RELATEDTABLE(Sales))
USERELATIONSHIP()	Activates an inactive relationship for a specific calculation.	CALCULATE(..., USERELATIONSHIP(Sales[ShipDate], Dates[Date]))
CROSSFILTER()	Changes the filter direction between two tables.	CALCULATE(..., CROSSFILTER(Sales[Customer], Customers[ID], BOTH))
UNION()	Combines rows from two tables into one table.	UNION(Table1, Table2)

Table 9: Relationship and Table Functions

5. CALCULATE Function - Deep Dive

CALCULATE is the most important function in DAX. Many interview questions are about CALCULATE. You must understand it very well. CALCULATE evaluates an expression (like SUM or AVERAGE) in a modified filter context. This means you can change what data is included in the calculation without changing the visual filter. For example, you can show total sales for a specific year, region, or product - all inside one measure.

Syntax

```
CALCULATE(expression, filter1, filter2, ...)
```

The first argument is the expression you want to calculate (like SUM(Sales[Amount])). The arguments after that are filter conditions. You can add as many filters as you need. All filters are combined with AND logic - meaning all conditions must be true for a row to be included.

Examples

```
Sales 2024 = CALCULATE(SUM(Sales[Amount]), Sales[Year] = 2024)
Red Products Sales = CALCULATE(SUM(Sales[Amount]), Products[Color] = "Red")
All Region Total = CALCULATE(SUM(Sales[Amount]), ALL(Sales[Region]))
```

In the first example, CALCULATE adds a filter for year 2024. In the second, it filters for red products. In the third, the ALL function removes the region filter, so you get the total across all regions regardless of what the user selected. This is very useful for calculating percentages or comparing filtered results to total results.

CALCULATE with Multiple Filters

```
Total = CALCULATE(SUM(Sales[Amount]),
    Sales[Year] = 2024,
    Sales[Region] = "Europe",
    Products[Category] = "Electronics"
)
```

In this example, all three filters must be true at the same time. Only sales from 2024, in Europe, and in the Electronics category will be included in the total. This is how AND logic works in CALCULATE. If you need OR logic, you can use the OR operator inside a single filter, or use the FILTER function for more complex conditions.

6. Filter Context and Row Context

Filter context and row context are two very important concepts in DAX. They are also two of the most difficult concepts for beginners. Interviewers ask about these to check how deeply you understand DAX. Let us explain each one in simple words.

Filter Context

Filter context means the filters that are currently active when a DAX expression is evaluated. These filters can come from many places: slicers on the report page, row and column headers in a matrix visual, the CALCULATE function, or filter pane selections. For example, if you have a slicer for "Year = 2024" and a bar chart shows sales by region, then the filter context for each bar includes both "Year = 2024" and the specific region for that bar. The filter context determines which rows of data are included in the calculation.

Row Context

Row context means DAX is evaluating a formula row by row, one row at a time. This happens when you create a calculated column or use certain functions like FILTER or SUMX. In row context, DAX knows which row it is on and can use the values from that specific row. For example, if you create a calculated column with the formula "Sales[Amount] * Sales[Qty]", DAX goes through each row and multiplies the amount and quantity for that row. It does not need any aggregation because it already knows which row to look at.

Key Difference

The main difference is: filter context determines WHICH rows to include (filtering), while row context determines which specific row DAX is looking at (iterating). CALCULATE can modify the filter context. But CALCULATE cannot directly change the row context. To use a value from row context inside a filter context, you need to use functions like EARLIER or variables (VAR/RETURN). Understanding the difference between these two contexts is essential for writing correct DAX formulas and passing interview questions.

Feature	Filter Context	Row Context
What it does	Filters which rows are included	Iterates row by row
Created by	Slicers, visuals, CALCULATE	Calculated columns, iterator functions
Functions	CALCULATE, ALL, FILTER	SUMX, AVERAGEX, FILTER
Example	Year = 2024 (from slicer)	Each row in a calculated column

Table 10: Filter Context vs Row Context

7. Iterator Functions (X Functions)

Iterator functions are DAX functions that end with the letter "X". They work differently from normal aggregation functions. While SUM adds all values in a column directly, SUMX goes through each row one by one, evaluates an expression for each row, and then adds the results. This is called row-by-row iteration. Iterator functions are very powerful because they let you do calculations that regular aggregation functions cannot do.

How Iterator Functions Work

An iterator function takes two arguments: a table and an expression. It goes through each row of the table, evaluates the expression for that row, and then aggregates the results. For example, SUMX(Sales, Sales[Price] * Sales[Qty]) goes through each row in the Sales table, multiplies Price by Qty for each row, and then adds all the results together. You cannot do this with a simple SUM because the Price * Qty calculation needs to happen at the row level first.

Function	Description	Example
SUMX()	Evaluates expression for each row and returns the sum of all results.	SUMX(Sales, Sales[Price] * Sales[Qty])
AVERAGEX()	Evaluates expression for each row and returns the average.	AVERAGEX(Sales, Sales[Amount] * 1.1)
MINX()	Returns the minimum value from row-level evaluations.	MINX(Products, Products[Price] * Products[Discount])
MAXX()	Returns the maximum value from row-level evaluations.	MAXX(Sales, Sales[Amount] / Sales[Qty])
COUNTX()	Counts non-blank results from row-level evaluations.	COUNTX(Sales, Sales[Amount])
CONCATENATEX()	Joins the results of row-level evaluations with a separator.	CONCATENATEX(VALUES(Products[Color]), Products[Color], ", ")
GENERATE()	Creates a new table by combining each row with a related table.	GENERATE(Regions, TOPN(3, Sales))

Table 11: Iterator Functions (X Functions)

8. Data Modeling in Power BI

Data modeling is the process of organizing your data tables and relationships so that your reports work correctly and perform well. A good data model is the foundation of every good Power BI report. Interviewers always ask about data modeling concepts because a bad model leads to wrong numbers and slow reports.

Star Schema

Star schema is the best practice for data modeling in Power BI. It has one central fact table (like Sales) connected to several dimension tables (like Date, Product, Customer, Region). The fact table contains the numbers (amounts, quantities, prices). The dimension tables contain the descriptions and attributes (product names, customer names, dates). The fact table is in the middle, and the dimension tables surround it like the points of a star - that is why it is called a star schema.

Relationship Cardinality

Cardinality describes how tables relate to each other. There are three main types: One-to-Many (1:N), Many-to-One (N:1), and One-to-One (1:1). The most common type is Many-to-One. For example, many sales records can belong to one customer - this is a Many-to-One relationship from Sales to Customer. One-to-One means each row in table A matches exactly one row in table B. Many-to-Many relationships should be avoided in Power BI because they can cause incorrect results and performance problems.

Relationship Cross Filter Direction

Cross filter direction determines how filters flow between tables. Single direction means filters flow from the "one" side to the "many" side. For example, selecting a customer in a slicer filters the sales data for that customer. Both directions means filters flow both ways - selecting a customer filters sales, and selecting a sale can filter customers. Both direction can cause ambiguity and performance problems, so single direction is the default and recommended setting for most relationships.

Concept	Description	Best Practice
Star Schema	One fact table in the center, dimension tables around it	Always use for Power BI models
Fact Table	Contains numbers: amounts, quantities, transactions	Keep it narrow and long
Dimension Table	Contains descriptions: names, categories, dates	Keep it wide and short
1:N Relationship	One dimension row matches many fact rows	Most common, use as default
1:1 Relationship	Each row matches exactly one row in other table	Use sparingly, for security
M:M Relationship	Many rows match many rows	Avoid if possible, use bridge table
Single Direction	Filters flow from "one" to "many"	Default and recommended
Both Direction	Filters flow both ways	Use only when necessary

Table 12: Data Modeling Concepts

9. Date Table - Why It Matters

A date table is a special table in your Power BI model that contains a continuous list of dates. It is required for time intelligence functions like TOTALYTD, SAMEPERIODLASTYEAR, and DATEADD. Without a proper date table, these functions will not work correctly. Many interview questions about time intelligence start with "Do you have a date table in your model?"

Requirements for a Date Table

- â■¢ Must have one column with Date data type (continuous dates, no gaps)
- â■¢ Must have a row for every date in your data range
- â■¢ Must be marked as "Date Table" in Power BI Desktop (Modeling > Mark as Date Table)
- â■¢ Should have columns like Year, Month, Quarter, Day of Week for filtering
- â■¢ Can be created automatically with CALENDAR() or CALENDARAUTO() DAX functions

How to Create a Date Table

You can create a date table using DAX. The easiest way is to use CALENDARAUTO(), which looks at all date columns in your model and creates a date table that covers the full range. Or you can use CALENDAR() to specify exact start and end dates. After creating the table, you add columns for Year, Month, Quarter, etc. using DAX functions like YEAR(), MONTH(), and FORMAT().

```
DateTable = CALENDAR(DATE(2020,1,1), DATE(2025,12,31))
DateTable[Year] = YEAR(DateTable[Date])
DateTable[Month] = FORMAT(DateTable[Date], "MMMM")
DateTable[Quarter] = "Q" & CEILING(MONTH(DateTable[Date])/3, 1)
```

10. Common Interview Questions and Answers

Below are the most frequently asked Power BI interview questions. Study each question and answer carefully. The answers are written in simple English (A1 level) so they are easy to understand and remember. Practice explaining these answers out loud before your interview.

Q1: What is Power BI?

Power BI is a business intelligence tool by Microsoft. It helps you connect to data, transform it, build data models, and create interactive reports and dashboards. It has three parts: Desktop (for building), Service (for sharing), and Mobile (for viewing on the go).

Q2: What is DAX?

DAX stands for Data Analysis Expressions. It is a formula language used in Power BI to create custom calculations. You can write measures, calculated columns, and calculated tables with

DAX. It is similar to Excel formulas but more powerful.

Q3: What is the difference between a calculated column and a measure?

A calculated column creates a new column in your table. It calculates a value for each row and stores the result. It runs once at data refresh. A measure does not create a column. It calculates a value dynamically when users interact with the report. It responds to filters and slicers. Measures are usually better for performance because they do not use memory for storage.

Q4: What is CALCULATE and why is it important?

CALCULATE is the most important DAX function. It evaluates an expression in a modified filter context. You can use it to add, remove, or change filters before a calculation. For example, CALCULATE(SUM(Sales[Amount]), Sales[Year]=2024) calculates total sales only for the year 2024, regardless of other filters on the report.

Q5: What is filter context?

Filter context is the set of filters that determine which rows are included in a DAX calculation. Filters can come from slicers, visual row/column headers, the CALCULATE function, or the filter pane. Filter context is what makes Power BI interactive - when you click a slicer, the filter context changes and all measures recalculate.

Q6: What is row context?

Row context is when DAX evaluates a formula one row at a time. This happens in calculated columns and iterator functions (like SUMX, FILTER). In row context, DAX knows which specific row it is working on and can use values from that row directly.

Q7: What is the difference between SUM and SUMX?

SUM adds up all values in a single column directly. SUMX goes through each row of a table, evaluates an expression for each row, and then adds up the results. Use SUM when you just need to add one column. Use SUMX when you need to do a row-level calculation first (like Price multiplied by Quantity) and then add the results.

Q8: What is a star schema?

A star schema is a data model design where one fact table (with numbers) is in the center, connected to several dimension tables (with descriptions). The dimension tables surround the fact table like points of a star. This is the best practice for Power BI models because it is simple, fast, and easy to understand.

Q9: What is a date table and why do you need it?

A date table is a table with one row for each date in your data range. It is required for time intelligence functions like TOTALYTD, SAMEPERIODLASTYEAR, and DATEADD. Without a date table, these functions do not work correctly. You must mark the table as "Date Table" in Power BI Desktop.

Q10: What is the difference between Power Query and DAX?

Power Query (M language) is for data transformation - cleaning, filtering, and shaping data before it loads into the model. It runs at data refresh time. DAX is for calculations after data is loaded - creating measures, calculated columns, and custom aggregations. DAX runs when users interact with the report. Use Power Query for data preparation and DAX for analysis and reporting.

Q11: What are inactive relationships and when do you use them?

In Power BI, a table can have multiple relationships between the same two tables, but only one can be active at a time. An inactive relationship is a relationship that exists but does not filter data by default. You activate it in a DAX formula using the USERELATIONSHIP function. This is useful when you have a fact table connected to a date table by both Order Date and Ship Date - one relationship is active, and the other is inactive.

Q12: What is the difference between DISTINCTCOUNT and COUNT?

COUNT counts all rows that have a value (ignores blanks but counts duplicates). DISTINCTCOUNT counts only unique values (ignores duplicates). For example, if a customer bought 5 times, COUNT gives 5 but DISTINCTCOUNT gives 1 for that customer.

Q13: How do you handle M:M (many-to-many) relationships in Power BI?

Many-to-many relationships can cause ambiguity and incorrect results. The best way to handle them is to create a bridge table (also called a junction table) in between. A bridge table breaks the M:M into two 1:N relationships. For example, Students and Classes have a M:M relationship, so you create an Enrollments table between them.

Q14: What is a KPI in Power BI?

A KPI (Key Performance Indicator) is a visual that shows progress toward a goal. It has three parts: an indicator value (the current number), a target value (the goal), and a status (good, bad, or neutral based on the comparison). KPIs help managers quickly see if they are on track to meet their targets.

Q15: What is incremental refresh and why is it useful?

Incremental refresh means refreshing only the new or changed data instead of the entire dataset. This makes refresh much faster for large datasets. For example, instead of refreshing 10 years of sales data every day, you only refresh the last 30 days and keep the rest in cache. This is very important for large models with millions of rows.

11. Best Practices for Power BI Development

Interviewers often ask about best practices because they want to know if you can build reports that are fast, correct, and easy to maintain. Below are the most important best practices that you should mention in your interview. These show that you have real experience and understand how to build professional Power BI solutions.

Data Model Best Practices

- â■¢ Always use a star schema with one fact table and dimension tables
- â■¢ Use single direction filter relationships (not both directions) unless you have a specific need
- â■¢ Avoid many-to-many relationships - use bridge tables instead
- â■¢ Create a separate date table and mark it as "Date Table"
- â■¢ Hide foreign key columns from the report view (they are not needed by users)
- â■¢ Name your tables and columns clearly so other developers can understand the model
- â■¢ Set the data type and format of each column correctly (currency, percentage, date, etc.)

DAX Best Practices

- â■¢ Use measures instead of calculated columns when possible (better performance)
- â■¢ Use variables (VAR/RETURN) to make complex DAX easier to read and faster
- â■¢ Avoid using FILTER on large tables inside CALCULATE when you can use simple filter arguments
- â■¢ Use DIVIDE() function instead of the / operator to handle division by zero
- â■¢ Write meaningful names for your measures (like "Total Sales Amount" not "Measure1")
- â■¢ Add comments to complex DAX formulas using // for single line or /* */ for multi-line

Report Design Best Practices

- â■¢ Put important KPIs at the top of the page where users see them first
- â■¢ Do not put too many visuals on one page - keep it clean and readable
- â■¢ Use consistent colors and fonts across all pages
- â■¢ Use bookmarks and buttons for navigation between pages
- â■¢ Add tooltips to give users extra information when they hover over visuals
- â■¢ Use page-level and report-level filters for common filtering needs
- â■¢ Test your report with real data and real users before publishing

12. Important DAX Patterns for Interviews

DAX patterns are common calculation patterns that you use again and again. Interviewers love to ask you to write DAX for common scenarios like percentage of total, year-over-year growth, running total, and ranking. Below are the most important patterns with their DAX code. Study these carefully and practice writing them.

12.1 Percentage of Total

This pattern calculates what percentage each category is of the total. It is one of the most common patterns. The key is using ALL to remove the category filter so you get the total, then dividing the current value by the total.

```
Pct of Total =
VAR CurrentSales = SUM(Sales[Amount])
VAR TotalSales = CALCULATE(SUM(Sales[Amount]), ALL(Products[Category]))
RETURN DIVIDE(CurrentSales, TotalSales)
```

12.2 Year-over-Year (YoY) Growth

This pattern compares sales this year to sales last year and calculates the growth percentage. It uses SAMEPERIODLASTYEAR to get last year dates and CALCULATE to compute last year sales. This is a very common interview question.

```
YoY Growth =
VAR CurrentSales = SUM(Sales[Amount])
VAR LastYearSales = CALCULATE(SUM(Sales[Amount]),
SAMEPERIODLASTYEAR(Dates[Date]))
RETURN DIVIDE(CurrentSales - LastYearSales, LastYearSales)
```

12.3 Running Total

A running total (also called cumulative total) shows the total from the beginning up to the current row. For example, running total of monthly sales shows total sales from January up to the current month. This pattern uses CALCULATE and DATESYTD or a custom filter.

```
Running Total YTD =
CALCULATE(
SUM(Sales[Amount]),
DATESYTD(Dates[Date])
)
```

12.4 Ranking Products or Customers

The RANKX function creates a ranking based on any measure. For example, you can rank products by total sales from highest to lowest. This is useful for top-N analysis and leaderboards.

```
Product Rank =
RANKX(
ALL(Products[ProductName]),
CALCULATE(SUM(Sales[Amount])),
, DESC
)
```

12.5 New vs Returning Customers

This pattern finds customers who bought for the first time in a period versus customers who bought before. It is a common business question. You use MIN to find the first order date for each customer and compare it to the current filter period.

```
New Customers =
VAR CurrentPeriodStart = MIN(Dates[Date])
VAR CurrentPeriodEnd = MAX(Dates[Date])
VAR FirstBuy =
CALCULATE(
MIN(Sales[OrderDate]),
ALLEXCEPT(Sales, Sales[CustomerID])
)
RETURN
COUNTROWS(
FILTER(
VALUES(Sales[CustomerID]),
FirstBuy >= CurrentPeriodStart && FirstBuy <= CurrentPeriodEnd
)
)
```

13. Quick Reference - Key Facts

This section gives you a quick summary of the most important facts. Use this as a cheat sheet before your interview. Read through all these facts to refresh your memory quickly.

Top 20 Facts to Remember

1. Power BI has three parts: Desktop, Service, and Mobile

2. DAX = Data Analysis Expressions (formula language for calculations)

3. M = Power Query language (for data transformation)

4. CALCULATE is the most important DAX function - it changes filter context

5. Measures are dynamic (recalculate with filters), calculated columns are static

6. Star schema = one fact table + many dimension tables

7. Filter context = which rows are included (from slicers, visuals, CALCULATE)

8. Row context = evaluating row by row (calculated columns, X functions)

9. SUMX iterates row by row and evaluates expression per row, SUM just adds a column

10. Date table is required for time intelligence functions

11. CALENDAR() and CALENDARAUTO() create date tables

12. TOTALYTD = year-to-date total, SAMEPERIODLASTYEAR = same period last year

13. ALL() removes filters, ALLEXCEPT() removes all except specified

14. RELATED() gets value from related table (many-to-one direction)

15. USERELATIONSHIP() activates an inactive relationship temporarily

16. DIVIDE() is safer than / because it handles division by zero

17. VAR/RETURN makes DAX easier to read and can improve performance

18. Single direction relationships are better than both direction (default recommendation)

19. Incremental refresh refreshes only new data, not the whole dataset

20. DISTINCTCOUNT counts unique values, COUNT counts all non-blank values

14. How Professionals Work with Power BI

When a professional Power BI developer starts a new project, they follow a clear and organized sequence of steps. This is called a workflow. If you follow the same workflow, you will work like a real expert. In interviews, if you explain these steps, the interviewer will see that you have practical knowledge and not just theory. Below is the exact workflow that professionals use in real companies.

The Professional Workflow (Step by Step)

Phase 1 - Understand the Business Requirements

Before you open Power BI, you must understand what the business needs. Meet with stakeholders (managers, directors, team leaders) and ask: What questions do you want to answer? What KPIs do you track? What decisions will you make from this report? Write down all requirements. This step is very important because a report that does not answer the right questions is useless, no matter how beautiful it looks. Professionals spend 20-30% of their project time on this phase alone.

Phase 2 - Collect and Explore the Data

Find out where the data comes from. Is it in SQL Server, Excel, SharePoint, or an API? Connect to the data sources and explore the tables and columns. Check data quality: Are there null values? Are there duplicates? Are dates in the correct format? Are column names clear? You must understand your data before you build anything. Write down the table names, column names, and relationships you find. Professionals call this step "data profiling" or "data discovery."

Phase 3 - Clean and Transform the Data (Power Query)

Open Power Query Editor and clean your data. Remove unnecessary columns and rows. Fix data types (change text dates to real dates, text numbers to real numbers). Split columns, merge columns, rename columns to clear names. Handle null values (replace blanks with 0 or remove rows). Merge queries when you need to join tables from different sources. This is where you do the heavy data cleaning work. Professionals call this step "ETL" (Extract, Transform, Load).

Phase 4 - Build the Data Model

Go to the Model view in Power BI Desktop. Create relationships between your tables. Build a star schema: one fact table in the center, dimension tables around it. Create a date table and mark it as "Date Table." Set the correct cardinality (1:N or N:1). Set filter direction (single is default). Hide foreign key columns from report view. Organize tables into display folders. Name your measures clearly. A good data model is the most important part of any Power BI project.

Phase 5 - Write DAX Measures

After the model is ready, write your DAX measures. Start with basic measures (Total Sales, Total Orders, Average Price). Then build more complex measures (YoY Growth, Running Total, Percentage of Total). Use VAR/RETURN for complex formulas. Test each measure by adding it to a table visual and checking the numbers. Compare your results with known numbers to verify correctness. Professionals always test their measures before building visuals.

Phase 6 - Design the Report Pages

Create report pages with a clear structure. Page 1 should be the main dashboard with key KPIs at the top. Add slicers for filtering (Year, Region, Category). Create charts and tables below. Use consistent colors and fonts. Add page navigation with bookmarks and buttons. Add tooltips for extra information. Make sure the report is clean, not crowded. Every visual should serve a purpose. If a visual does not add value, remove it.

Phase 7 - Test and Validate

Before publishing, test everything. Check that all numbers are correct. Ask a colleague or stakeholder to review the report. Test slicers, drill-down, cross-filter, and bookmarks. Check performance - does the report load fast? Check on mobile - does it look good on a phone? Fix any issues you find. This quality check step separates amateurs from professionals.

Phase 8 - Publish and Share

Publish the report to Power BI Service. Create a workspace for the project. Set up scheduled data refresh (daily, weekly, or monthly). Set row-level security (RLS) so users only see data they are allowed to see. Share the report with stakeholders. Create dashboards from the most important report visuals. Set up email subscriptions for automatic report delivery.

Phase 9 - Maintain and Improve

After publishing, the work is not finished. Monitor report usage - which pages are viewed most? Get feedback from users. Fix bugs that users report. Add new features when business needs change. Optimize slow reports. Update data sources when systems change. A Power BI report is a living product that needs continuous care and improvement.

Phase	Step	Tool / View	Time (%)
1	Understand Requirements	Meetings, Documents	20-30%
2	Explore Data	Power BI (Get Data)	5-10%
3	Clean and Transform	Power Query Editor	15-20%
4	Build Data Model	Model View	10-15%
5	Write DAX Measures	Modeling Tab	10-15%

6	Design Report	Report View	15-20%
7	Test and Validate	Report View	5-10%
8	Publish and Share	Power BI Service	2-5%
9	Maintain and Improve	Ongoing	Ongoing

Table 13: Professional Power BI Workflow Phases

15. Step-by-Step Tutorial: Install and Start Learning

This section is your personal learning guide. Follow every step in order. By the end, you will have Power BI installed, loaded with practice data, and you will have built your first report. Think of this as following an expert who is showing you exactly what to do. Take your time with each step and do not skip ahead.

Step 1: Install Power BI Desktop (Free)

Power BI Desktop is free. You do not need to pay anything. Follow these steps to install it on your Windows computer. Note: Power BI Desktop only works on Windows (Windows 10 or Windows 11). If you have a Mac, you can use Power BI Service in your browser, or install Windows on a virtual machine.

1. Open your web browser and go to: https://powerbi.microsoft.com/en-us/desktop/

2. Click the big "Download Free" button on the page.

3. The download will start. The file is about 800 MB, so it may take 5-15 minutes depending on your internet speed.

4. When the download finishes, open the file (it is called "PBIDesktopSetup.exe").

5. Click "Next" on the setup wizard. Accept the license terms.

6. Choose the installation folder (the default is fine - just click Next).

7. Click "Install" and wait for the installation to finish (2-5 minutes).

8. Click "Finish" when the installation is complete.

9. Open Power BI Desktop from your Start Menu. You will see the welcome screen with "Get Data", "Recent", and "Open" options.

10. You are ready! Power BI Desktop is now installed and free to use forever.

Step 2: Download Practice Data

To learn Power BI, you need data to practice with. The best free practice data is the Adventure Works sample database. This is the same data that Microsoft uses in all official tutorials. It has sales data, customer data, product data, and date data - everything you need to practice all the topics in

this guide. Go to Section 16 in this document for more data sources and download links.

Step 3: Load Data into Power BI

1. Open Power BI Desktop. On the start screen, click "Get Data" (or click "Get Data" on the Home tab).

2. In the "Get Data" window, click "Excel workbook" and then click "Connect".

3. Browse to the folder where you saved the Excel file and select it. Click "Open".

4. In the Navigator window, select the tables you want to load (select all of them for practice).

5. Click "Load" at the bottom of the Navigator window.

6. Wait a few seconds. The tables will appear in the Fields pane on the right side of the screen.

7. Click the Data icon (table icon) on the left to see your loaded data in a table format.

8. Click the Model icon (relationship icon) on the left to see how your tables are connected.

Step 4: Explore the Power BI Interface

Before building a report, spend 10-15 minutes exploring the Power BI Desktop interface. There are three main views that you need to know. On the left side of the screen, you will see three icons at the bottom: the Report view (looks like a chart), the Data view (looks like a table), and the Model view (looks like a diagram). The Report view is where you build visualizations. The Data view is where you see your data rows and columns. The Model view is where you create relationships between tables. On the right side, you will see the Fields pane (list of all your tables and columns), the Visualizations pane (icons for all available chart types), and the Filters pane.

Step 5: Build Your First Report

1. Make sure you are in the Report view (click the chart icon on the left).

2. Click the "Stacked column chart" icon in the Visualizations pane. An empty chart appears on the canvas.

3. In the Fields pane, open your Sales (or Fact) table. Drag the "Amount" column to the "Y axis" field.

4. Drag the "Date" or "Year" column to the "X axis" field. Your chart now shows data!

5. Click the "Map" visual icon. Drag "City" to Location and "Sales Amount" to Size. You now have a map.

6. Click the "Card" visual icon. Drag "Total Sales" measure (or SUM of Amount) to the Fields box.

7. Resize your visuals by dragging their edges. Move them by dragging their title bar.

8. Click "File" and then "Save" to save your first Power BI report. Congratulations!

Step 6: Practice DAX with Your Data

Now that you have data loaded, practice writing DAX measures. Go to the Modeling tab and click "New Measure." A formula bar appears at the top of the screen. Try writing these measures one by one. After each measure, check it by adding a Card visual to your report page.

```
Total Sales = SUM(Sales[Amount])

Total Orders = COUNTROWS(Sales)

Average Sale = AVERAGE(Sales[Amount])

Sales 2024 = CALCULATE(SUM(Sales[Amount]), Sales[Year] = 2024)

Pct of Total = DIVIDE(SUM(Sales[Amount]), CALCULATE(SUM(Sales[Amount]),
ALL(Sales[Region])))
```

Step 7: Your Weekly Learning Plan

To learn Power BI well, you need a plan. Do not try to learn everything in one day. Here is a recommended weekly plan that follows the professional workflow from Section 14. Follow this plan and you will be ready for interviews in 4-6 weeks.

Week	Focus Topic	Practice Activity	Time
Week 1	Interface and Data Loading	Install Power BI, load Excel data, explore all views	5-7 hours
Week 2	Power Query and Cleaning	Clean messy data: remove nulls, merge tables, change types	5-7 hours
Week 3	Data Modeling	Create star schema, build relationships, date table	5-7 hours
Week 4	DAX Basics	Write measures: SUM, CALCULATE, FILTER, ALL	7-10 hours
Week 5	DAX Advanced	Time intelligence, iterators, VAR/RETURN	7-10 hours
Week 6	Report Design	Build full dashboard: KPIs, slicers, bookmarks	5-7 hours

Table 14: Weekly Learning Plan

16. Where to Find Realistic Practice Data

One of the biggest challenges when learning Power BI is finding good practice data. You need data that looks real and has enough tables and rows to practice all the concepts: relationships, DAX, time intelligence, and visualizations. Below is a list of the best free data sources that professionals recommend. Each source is described in detail so you know what to expect before you download it.

1. Microsoft Adventure Works (Highly Recommended)

This is the official Microsoft sample database. It is the best dataset for learning Power BI because it is specifically designed for this purpose. It contains realistic data about a bicycle company: sales, customers, products, territories, and dates. The data has multiple related tables, which is perfect for practicing star schema, relationships, and DAX calculations. There are several versions: Adventure Works DW (data warehouse) and Adventure Works LT (lightweight). The data warehouse version is better because it already has a fact table and dimension tables. You can download it as an Excel file or as a SQL Server backup. For beginners, the Excel version is easiest - just open it in Power BI and start practicing. Search for "Adventure Works Power BI sample" on Google.

2. Microsoft Sample Datasets in Power BI

Power BI Desktop has built-in sample datasets. When you open Power BI, click "Try a sample dataset" on the welcome screen. You will see several options: Financial Sample, HR Analytics Sample, Sales Analysis Sample, and Customer Profitability Sample. These samples already have data models, measures, and report pages built by Microsoft. You can open them, study how they are built, and modify them. This is a great way to learn by looking at expert work. The Sales Analysis sample is the most useful because it has sales, products, stores, and date data.

3. Kaggle Datasets (kaggle.com)

Kaggle is a website owned by Google where data scientists share datasets. It has thousands of free datasets in many categories: finance, sales, healthcare, sports, weather, and more. You can download any dataset as a CSV file and load it into Power BI. Some popular datasets for Power BI practice include: "Superstore Sales" (retail sales with products, customers, and dates), "Online Retail" (e-commerce transactions from a UK-based store), "HR Analytics" (employee data with salaries, departments, and performance scores), and "COVID-19 Data" (country-level statistics over time). Go to kaggle.com and search for "Power BI dataset" or "sales dataset" to find good options. You need to create a free account to download datasets.

4. World Bank Open Data (data.worldbank.org)

The World Bank provides free data about countries around the world. This data includes GDP, population, education, health, and many other indicators for every country. It is perfect for practicing maps, comparisons between countries, and time series analysis. You can download the data as CSV or Excel files, or connect directly from Power BI using the "Web" data source. This data is updated regularly, so you can practice with fresh data. Search for "World Bank Open Data" on Google to find the download page.

5. Gapminder (gapminder.org/data)

Gapminder provides free data about global development. It has data on health, wealth, population, education, and environment for all countries over many years. The famous "Gapminder bubble

chart" shows life expectancy vs income per person for all countries. You can download the data and try to recreate this chart in Power BI. It is a great exercise for scatter/bubble chart practice. The data is available in Excel and CSV formats.

6. Create Your Own Data (Excel Practice)

You can also create your own practice data in Excel. This is actually one of the best ways to learn because you understand the data completely. Create a simple sales dataset with 4 tables: a Sales table (OrderID, Date, CustomerID, ProductID, Amount, Qty), a Customers table (CustomerID, Name, City, Country, Region), a Products table (ProductID, ProductName, Category, Price), and a Dates table (Date, Year, Month, Quarter). Enter 50-100 rows of data. Save it as an Excel file and open it in Power BI. Then practice building relationships, writing DAX measures, and creating visualizations with your own data. This gives you full control and deep understanding.

7. GitHub - Awesome Public Datasets

GitHub has a popular repository called "awesome-public-datasets" that lists hundreds of free datasets organized by category. Go to github.com/awesomedata/awesome-public-datasets to see the full list. Categories include Agriculture, Biology, Climate, Economics, Education, Finance, Healthcare, Sports, and Transportation. Each dataset has a link, description, and format (CSV, JSON, SQL, etc.). This is a great resource when you want to practice with data from a specific industry or domain.

Data Source	Format	Difficulty	Best For
Adventure Works	Excel / SQL	Beginner-Friendly	Star schema, DAX, relationships
Power BI Built-in Samples	Direct in PBI	Beginner	Learning from expert examples
Kaggle (Superstore)	CSV	Beginner	Sales analysis, E-commerce
World Bank Open Data	CSV / API	Intermediate	Maps, country comparisons
Gapminder	Excel / CSV	Beginner	Bubble charts, global data
Your Own Excel	Excel	Beginner	Full control, deep understanding
GitHub Awesome Datasets	Various	All Levels	Industry-specific practice

Table 15: Free Data Sources for Power BI Practice